QUIZ PERTEMUAN 4

PEMROGRAMAN BERORIENTASI OBJEK

Oleh:

Zeyan Fahluzi Alhariz

250660221033

SOAL

1. Superclass dan Subclass

Jelaskan perbedaan superclass dan subclass. Berikan satu contoh hierarki class dari sistem informasi perpustakaan.

Jawaban:

- Perbedaan:
- Superclass (Kelas Induk): Kelas dasar yang mewariskan atribut dan *method*-nya ke kelas lain. Kelas ini berisi karakteristik umum.
- Subclass (Kelas Anak): Kelas turunan yang mewarisi sifat dari *superclass*, namun dapat memiliki atribut atau *method* tambahan yang lebih spesifik, serta dapat memodifikasi *method* bawaan induknya.
- Contoh Hierarki (Sistem Informasi Perpustakaan):
- Superclass: `KoleksiPerpustakaan` (memiliki atribut umum seperti `id_item`, `judul`, `tahun_terbit`).
- Subclass: `Buku` (menambahkan atribut `penulis`, `isbn`) dan `Majalah` (menambahkan atribut `edisi`, `nomor_volume`).

2. Fungsi super().__ init __()

Jelaskan fungsi super().__ init __() pada constructor subclass dari sistem informasi perpustakaan.

Jawaban:

- Fungsi: Perintah `super().__init__()` digunakan di dalam *constructor* (method `__init__`) milik *subclass* untuk memanggil *constructor* milik *superclass*.
- Alasan Diperlukan: Pemanggilan ini diperlukan agar semua atribut yang didefinisikan di dalam *superclass* dapat diinisialisasi dan diwariskan ke objek *subclass* dengan benar sebelum *subclass* tersebut menambahkan atribut spesifiknya sendiri. Jika tidak dipanggil, atribut dari *superclass* tidak akan tersedia di *subclass*, sehingga dapat menyebabkan *error* saat diakses.

3. Overriding

Jelaskan overriding dan berikan contoh penerapannya pada method yang diwarisi dari superclass.

Jawaban:

- Penjelasan Overriding: Konsep di mana sebuah *subclass* mendefinisikan ulang (menimpa) implementasi dari sebuah *method* yang sudah ada dan diwarisi dari *superclass*-nya. Hal ini dilakukan agar *method* tersebut memiliki perilaku yang lebih spesifik sesuai dengan kebutuhan *subclass*.
- Contoh Penerapan:
- *Superclass* `Karyawan` memiliki *method* `hitung_gaji()` yang mengembalikan gaji pokok.
- *Subclass* `Manajer` mewarisi `Karyawan` namun melakukan *overriding* pada `hitung_gaji()` untuk mengembalikan hasil penjumlahan gaji pokok ditambah tunjangan jabatan.

4. Hubungan is-a

Mengapa inheritance sebaiknya digunakan ketika terdapat hubungan is-a? Jelaskan alasan teknisnya.

Jawaban:

- Alasan Penggunaan Inheritance untuk "is-a": *Inheritance* (pewarisan) secara semantik dirancang untuk memodelkan hubungan "adalah sebuah" (*is-a*), misalnya "Mobil *adalah sebuah* Kendaraan" atau "Buku *adalah sebuah* Koleksi".
- Alasan Teknis: Mematuhi hubungan *is-a* memastikan validitas Prinsip Substitusi Liskov (*Liskov Substitution Principle*), yang berarti objek dari *subclass* harus selalu bisa menggantikan objek *superclass* tanpa merusak logika program. Jika *inheritance* dipaksakan hanya untuk menggunakan ulang kode padahal tidak ada hubungan *is-a* (misalnya memaksakan kelas `Mesin` menjadi induk dari `Mobil`), hal ini akan menghasilkan arsitektur kode yang kaku, *method* yang tidak relevan ikut terwarisi, dan sistem yang rentan terhadap *bug* (sebaiknya gunakan komposisi / *has-a* untuk kasus.

5. Prediksi Output

```
class Pengguna:
    def tampilkan_peran(self):
        return "Pengguna"

class Mahasiswa(Pengguna):
    def tampilkan_peran(self):
        return "Mahasiswa"
```

Tuliskan Output dari Mahasiwa().tampilan_peran() dan jelaskan konsep OOP yang ditujukan.

Jawaban:

Output yang muncul:

Mahasiswa

Konsep OOP yang Ditunjukkan: Kode tersebut menunjukkan konsep Overriding (yang juga merupakan bentuk dari Polymorphism). *Subclass* `Mahasiswa` menimpa *method* `tampilkan_peran()` milik *superclass* `Pengguna`, sehingga ketika *method* tersebut dipanggil dari instansiasi `Mahasiswa()`, yang dieksekusi adalah implementasi spesifik milik `Mahasiswa` (mengembalikan "Mahasiswa"), bukan milik `Pengguna`.